

Laboratório
Concorrente

de

Programação

Lab11 - 26.1

Coordenação de
Goroutines com
Select e Canais

Objetivos

Após este roteiro prático, você deverá ser capaz de:

- revisar o uso de canais bidirecionais e unidirecionais;
- compreender quando utilizar canais bufferizados;
- utilizar a instrução `select`;
- receber mensagens provenientes de múltiplos canais;
- utilizar `default` em estruturas `select`;
- construir aplicações concorrentes que coordenam diversas goroutines.

Contexto

No laboratório anterior estudamos:

- goroutines;
- canais;
- sincronização por comunicação;
- canais bufferizados;
- canais unidirecionais;
- pipelines simples.

Neste laboratório estudaremos como coordenar diversas goroutines simultaneamente. Para isso utilizaremos uma das estruturas mais importantes da linguagem Go:

```
select
```

Enquanto um `if` escolhe entre expressões booleanas, o `select` escolhe entre operações de comunicação realizadas através de canais.

1. Revisando canais unidirecionais

Passo 1

Implemente o código abaixo.

```
package main
import "fmt"

func produtor(c chan int) {
    c <- 10
}

func consumidor(c chan int) {
```

```

        valor := <-c
        fmt.Println(valor)
    }

    func main() {
        canal := make(chan int)
        go produtor(canal)
        consumidor(canal)
    }

```

Execute e observe que o programa funciona normalmente.

Passo 2

Agora altere apenas a assinatura da função produtora.

```
func produtor(c chan<- int)
```

Execute novamente. O comportamento mudou?

Passo 3

Faça o mesmo para o consumidor.

```
func consumidor(c <-chan int)
```

Passo 4

Tente modificar o produtor. O compilador aceita?

```
func produtor(c chan<- int){
    valor := <-c
    fmt.Println(valor)
}

```

Agora tente modificar o consumidor. O compilador aceita?

```
func consumidor(c <-chan int){
    c <- 20
}

```

Exercício de Codificação (main1.go)

Altere o programa para que:

- o produtor envie cinco números;
- o consumidor imprima todos eles;
- todas as funções utilizem canais unidirecionais.

Discussão (comments1.txt)

- Qual a vantagem dos canais unidirecionais?

- Eles alteram o comportamento do programa?
- O que muda é em tempo de compilação ou execução?

2. Comparando canais bufferizados

Passo 1

Implemente o código, execute e observe a saída.

```
package main

import (
    "fmt"
    "time"
)

func produtor(c chan<- int){
    for i:=1;i<=5;i++){
        fmt.Println("Enviando",i)
        c<-i
    }
}

func consumidor(c <-chan int){
    for i:=1;i<=5;i++){
        time.Sleep(time.Second)
        fmt.Println("Recebido",<-c)
    }
}

func main(){
    canal:=make(chan int)
    go produtor(canal)
    consumidor(canal)
}
```

Passo 2

Altere apenas uma linha e execute novamente.

```
canal:=make(chan int, 5)
```

Passe 3

Compare as duas execuções, pensando sobre as seguintes questões:

- Produtor bloqueia?
- Consumidor bloqueia?
- Tempo total aproximado

Exercício de Codificação (main2.go)

Implemente um produtor que envie **20 números**. Teste a execução com buffers com os seguintes tamanhos:

0
1
5
10
20

Discussão (comments2.txt)

Existe diferença perceptível? Se sim, quais?

3. Primeiro select

Até agora sempre recebemos dados de um único canal. Mas e se existirem vários produtores?

Go resolve isso utilizando o comando `select`. Este tem por objetivo receber mensagens provenientes de múltiplos canais.

Passo 1

Implemente o código abaixo e execute pelo menos 10 execuções. Observe quantas vezes cada mensagem apareceu primeiro.

```
package main

import (
    "fmt"
)

func produtorA(c chan<- string){
    c<-"Mensagem A"
}
```

```

func produtorB(c chan<- string){
    c<-"Mensagem B"
}

func main(){
    canalA:=make(chan string)
    canalB:=make(chan string)

    go produtorA(canalA)
    go produtorB(canalB)

    select{
    case msg:=<-canalA:
        fmt.Println(msg)

    case msg:=<-canalB:
        fmt.Println(msg)
    }
}

```

Exercício de Codificação (main3.go)

Modifique o programa para que:

- produtor A envie seu nome cinco vezes;
- produtor B envie seu nome cinco vezes.

Depois utilize um laço para receber dez mensagens utilizando apenas um select.

Discussão (comments3.txt)

- O select escolhe sempre o mesmo canal? Por quê?

4. Select contínuo

O objetivo é receber continuamente mensagens de vários produtores.

Código Inicial

Crie dois produtores. Cada um deverá executar:

```

for i:=1;i<=10;i++){
    c<-fmt.Sprintf("%s -> %d",nome,i)
}

```

Na função principal utilize.

```
for i:=0;i<20;i++){
    select{
        case msg:=<-canalA:
            fmt.Println(msg)

        case msg:=<-canalB:
            fmt.Println(msg)
    }
}
```

Exercício de Codificação (main4.go)

Adicione um terceiro produtor sem modificar os produtores anteriores e adaptando o select.

Cada produtor deverá dormir um tempo diferente.

```
300 ms
600 ms
1000 ms
```

Discussão (comments4.txt)

Execute o código, observe como a ordem muda e justifique esse comportamento.

5. Utilizando default

Nem sempre queremos bloquear uma goroutine. O uso de default está direcionado para evitar bloqueios.

Código Inicial

```
select{
    case msg:=<-canal:
        fmt.Println(msg)
    default:
        fmt.Println("Nenhuma mensagem")
}
```

Passo 1

Execute o código acima antes do produtor iniciar a inserir dados no canal. Em seguida, execute após o produtor enviar. Observe o comportamento.

Exercício de Codificação (main5.go)

Considere o código produzido no [main4.go](#) e adapte a solução de forma a implementar o seguinte comportamento: Enquanto a primeira mensagem não chegar chegar, imprima:

Aguardando...

Assim que receber a primeira mensagem, interrompa o laço.

Discussão (comments5.txt)

- Quando utilizar default? Quando ele pode desperdiçar processamento?

Entrega

A entrega se dará através das atualizações do código no próprio repositório no Github Classroom. Você deve submeter os seis arquivos main mencionados anteriormente.

Exemplos

Gerar Número Aleatório em Go

```
import (  
    "fmt"  
    "math/rand"  
    "time"  
)  
  
func main() {  
    // A partir do Go 1.20, a semente (seed) é inicializada automaticamente.  
    // Em versões anteriores, era necessário usar  
    rand.Seed(time.Now().UnixNano())  
  
    // Gera um número inteiro de 0 até 99  
    numero := rand.Intn(100)  
    fmt.Println("Número aleatório:", numero)  
}
```

Sleep em Go

```
package main  
  
import (  
    "fmt"  
    "time"  
)  
  
func main() {  
    fmt.Println("Início...")  
  
    // Pausa a execução por 2 segundos  
    time.Sleep(2 * time.Second)  
  
    fmt.Println("Fim após 2 segundos!")  
}
```